UFUG2106 ·Round 2 Local Reproduction Report

Hybrid BDD–Implicant Synthesis ·Five-candidate Portfolio & Comparative Study

UFUG2106 ·Truth Table to Circuit ·Local Reproduction Archive

2026-11-11

目录

1. 研究问题与成本模型 1
2. Round 2：算法设计与数学原理 2
3. 实验环境与可追溯性 4
4. 官方测评与同机对照 5
5. 验证覆盖、证据链与可信度边界 7
6. Round 2 的局限性及向 Round 3 的演进 8
7. 复现命令、版本边界与证据索引 8

文档性质：基于用户于 2026-11-11 上传的 WSL 本地复现 ZIP 中的原始日志、Evaluator CSV、Netlist、源码快照，以及与该版本对应的算法设计文档。本文是已核验的技术记录，不代表官方最终评分、全局最优性证明或隐藏测试表现。算法的数学正确性说明与实际运行的经验事实严格区分。

1. 研究问题与成本模型

完整真值表描述的是一个多输出 Boolean Function：

$$F : \{0,1\}^n \longrightarrow \{0,1\}^m, \qquad F(x) = (f_0(x), \dots, f_{m-1}(x)).$$

题目要求把它实现为有限、有向无环的门级 Netlist。只允许 **AND2、OR2、NOT** 三类门；编号 0 表示常量 0，编号 1 到 n 是输入变量 $x_0, \dots, x_{n-1}$，新门的输入只能引用更早的信号。一个输出可以直接指向输入或常量，不必多造一层门。真值表按 MSB-first 的变量顺序枚举所有 2^n 行。

对电路 C，使用当前测试案例的门成本 a_g, d_g：

$$A(C) = \sum_{g \in G(C)} a_g, \qquad T(v) = \max_{u \in \mathrm{fanin}(v)} T(u) + d_v, \qquad D(C) = \max_{j=0}^{m-1} T(y_j).$$

输入和常量的到达时间为 0；单输入 NOT 门只取其唯一输入的到达时间。共享 Gate 只计算一次 Area。$D(C)$ 是 **Circuit Delay**，程序执行所需的 **Runtime** 是另一个独立指标。老师的 `status=PASS` 与 `target_met=yes` 也是两个不同字段：正确电路可以超过 Target Delay 而仍得到 PASS。

本研究的核心目标是在满足 $C(x) = F(x)$ 的前提下探索低 Area、低 Delay 与受控 Runtime；并未假定老师采用跨案例 Area 简单相加作为成绩。

2. Round 2：算法设计与数学原理

2.1 为什么需要不同的 Function Representation?

Round 1 的固定顺序 BDD 保证正确性，却把电路限制在一种递归 Shannon MUX 结构中。低节点数的 BDD 也不必然给出最低成本的 AND2/OR2/NOT Netlist。Round 2 因此生成若干等价候选 $C_1, \dots, C_K$，再用完整 Netlist 的最终 Area / Delay 作选择：

$$\mathcal{K}(F) = \{C_1, \dots, C_K\} \subseteq \mathcal{C}(F), \quad \mathcal{C}(F) = \{C : C(x) = F(x),\ \forall x\}.$$

注意，这个包含关系依赖候选构造实现没有 Bug；理论上的等价性仍须通过测试检验具体程序。

2.2 变量影响度：为 BDD 搜索另一种顺序

定义变量 x_i 在多输出函数上的影响度：

$$I_i(F) = \sum_{j=0}^{m-1} \sum_{x:\, x_i=0} \mathbf{1}\left[f_j(x) \neq f_j(x \oplus e_i)\right],$$

其中 e_i 表示只有第 i 位为 1 的位向量。源码遍历两个赋值仅在 x_i 上不同的输入行，累计输出位变化次数，再按影响度降序尝试变量顺序；另一候选保留自然顺序。

变量顺序改变 BDD 共享结构和节点数量，却不改变 F 的语义。源码通过映射回原始输入行下标读取真值表，避免重新排序后把函数输入位解释错。

非最优性：影响度高并不等价于“应先测试”；本方法只是一个可计算的候选生成启发式。

2.3 两层 Shannon Expansion

按根变量 x 及两个分支各自的下一个变量 u, v 展开：

$$\begin{aligned} f &= \neg x\, f_0 \vee x\, f_1, \\ f_0 &= \neg u\, f_{00} \vee u\, f_{01}, \\ f_1 &= \neg v\, f_{10} \vee v\, f_{11}. \end{aligned}$$

严格代入可得:

$$\boxed{\begin{aligned} f = {} & (\neg x \wedge \neg u \wedge f_{00}) \vee (\neg x \wedge u \wedge f_{01}) \\ & \vee (x \wedge \neg v \wedge f_{10}) \vee (x \wedge v \wedge f_{11}). \end{aligned}}$$

两个 BDD 分支可能跳过不同变量, 所以 u 与 v 不要求相等。常量分支和叶子可以直接简化。四个 Product Terms 可以部分并行计算, 避免完全串行的两个 MUX Levels, 因此**可能**缩短关键路径, 但共享减少也可能增加 Gate 数量。

源码同时构造原映射和展开候选, 以局部 Arrival Time 和配置的 `margin` 判定是否接受展开; 这不是“展开必然更快”的定理。

2.4 到达时间驱动的平衡树

当多个 AND 或 OR Terms 需要合并时, 源码用 Priority Queue 反复选择到达时间较早的两个信号, 计算:

$$T_{\text{new}} = \max(T_a, T_b) + d_{\text{gate}}.$$

这种合并策略倾向于让较晚到达的子信号通过较少层级。它是局部 Arrival-time Heuristic; 即使所有门功能相同, 也不能简单推广成共享 DAG 上的全局最优证明。

2.5 De Morgan 与动态 Gate Costs

$$a \vee b = \neg(\neg a \wedge \neg b), \qquad a \wedge b = \neg(\neg a \vee \neg b).$$

例如, 如果所有三个新 NOT 都需要建立, 则右侧实现 OR 的 Area 上界为 $a_{\text{AND2}} + 3a_{\text{NOT}}$。在共享电路中, 某些取反信号可能已经存在, 真实增量 Area 需要依赖完整 Netlist 的可达门集合, 而非简单逐项相加。

Round 2 通过 `fast` 与 `rewrite` 两类映射配置, 选择在局部 Arrival Time 或 Gate Cost 条件下采用等价转换; 代价可能是增加临时门, 最后由 Dead-gate Elimination 清理。

2.6 Essential Support 与 Implicant Cover

变量 x_i 对 f_j 是 Essential, 当且仅当 $f_j|_{x_i=0} \not\equiv f_j|_{x_i=1}$。记该变量集合为 $\text{supp}(f_j)$。源码仅在 $n \le 13$ 且一个输出的 $|\text{supp}(f_j)| \le 9$ 时额外尝试 Implicant Cover; 其他情况仍有通用 BDD 候选作为保证。

程序从 ON-set 的 Minterms 出发, 合并只相差一个相反文字的 Terms, 形成更大的 Cubes; 再执行 Greedy Cover。若选中的 Cubes $Q_1, \dots, Q_r$ 满足

$$\bigcup_{k=1}^{r} Q_k = f_j^{-1}(1), \qquad Q_k \subseteq f_j^{-1}(1),$$

则它们的析取与 f_j 完全相等: 对 ON-set 的每个赋值至少一个 Cube 为 1, 而对 OFF-set 的每个赋值没有 Cube 为 1。对 OFF-set 建立 $\neg f_j$ 的 SOP 后再取反，可得到另一个 POS 风格候选。Greedy Cover 不保证最少文字或全局最低 Gate Area，但只要覆盖合法，其布尔等价性不依赖贪心的选择次序。

2.7 五候选 Portfolio 与预算限制

本归档 `src/synth.cpp` 实际包含以下**五种顺序执行的**配置:

候选	BDD Order	Gate Mapping	Two-level	Cubes
1	Influence-desc	`fast` delay-first	Margin 0	开启
2	Influence-desc	`rewrite` area-focused	Margin 0	开启
3	Influence-desc	`fast` delay-first	Margin 0	关闭
4	Natural	`fast` delay-first	Margin 0	开启
5	Influence-desc	`fast` delay-first	Margin 4	开启

对每个完成的候选，程序先重新解析整张 Netlist 并计算 Area 和 Delay，按如下**内部工程排序**选取:

$$\text{rank}(C) = \Big(\max(0, D(C) - D_{\text{target}}),\ \mathbf{1}_{D(C)\le D_{\text{target}}} A(C),\ D(C),\ A(C)\Big)$$

并使用字典序比较。该排序**不是老师发布的评分公式**。尤其要注意，候选超过 Target 时第二项为 0，使优先级先落在超出量，符合“先恢复 Timing Feasibility”的设计意图。

候选 1 必须先构造；从候选 2 开始，程序在**候选之间**检查是否超过 0.75 倍的输入 Runtime Limit。它不支持在一个复杂候选的内部强制抢占，因此在更严格的未知测试中仍可能超时。

2.8 跨算法的整体正确性证明

BDD 变量排序只改变 Cofactor 的展开次序,且索引映射保证原始真值表语义;Shannon 两层展开和 De Morgan 均为 Boolean 恒等式; Cubes 覆盖完整 ON-set 或 OFF-set 是严格的覆盖等价; Structure Hashing 和 Dead-gate Elimination 不改变输出函数；最终 Portfolio 只从这些完整候选中选择一个。因此，对程序**正常构造出的合法 Netlist**，有 $C(x) = F(x)$ 对全部输入赋值成立。

这一推导证明的是算法的函数保持性质，而非其全局面积最优或资源限制下的必然终止。

3. 实验环境与可追溯性

日期与证据包： `2026-11-11T10:40:26+08:00`; `20261111_104026_Round2_local_results.zip`。

项目	本地记录
操作系统	Ubuntu 24.04.4 LTS / WSL2；Linux `6.6.87.2-microsoft-standard-WSL2`, x86_64

项目	本地记录
CPU	Intel Core Ultra 9 275HX；WSL 报告 24 个逻辑 CPU；综合过程为单线程
编译器	GCC 13.3.0；C++17；`g++ -O2 -std=c++17 -Wall -Wextra`
Python	3.12.3，执行独立验证及汇总
数据路径	Windows `E:` 盘对应的 WSL `/mnt/e/` 挂载目录
Evaluator	老师提供的 Linux Evaluator，复制到 `/tmp/ufug2106_evaluator` 执行
源码 SHA-256	`f9fd99fe1f17849e33bc018b658000b313aabbbfd3833b11e1301`
Evaluator SHA-256	`142361d7ce18bc47dea6a07538db79a588b3afbf446526c1d1246`
原始 ZIP SHA-256	`d74c479ce5838cd45e42f6d6f42db2db15e37605626aa9c3e1ad0`

源码文件 `synth_source_snapshot.cpp` 与本轮冻结的 `src/synth.cpp` 经字节级 SHA-256 对照完全一致。报告复核 `official.csv` 中每个案例的 `status`、`area`、`delay`、`target_met`，并与同 ZIP 所含历史 `reference.csv` 比较。所有 25 行的 Area、Delay 与 Target Met 均保持一致；Runtime 不能要求逐字相等。

编译日志中的 `-Wmisleading-indentation` 警告指向一行多个 `if` 的排版，**不是编译失败**。这类警告仍值得在后续清理中修正，但本轮为了保持冻结版本不改动源文件。

4. 官方测评与同机对照

4.1 本轮概要

测评指标	本地测量	历史参考	解释
Official PASS	**25/25**	25/25	功能与格式通过老师的 Evaluator
Target Delay Met	**25/25**	25/25	`target_met` 单独统计，不等于 PASS
Area Sum	**790,243**	790,243	跨测试案例的描述性总和
Median Runtime	**36.427 ms**	55.460 ms	受主机与运行负载影响
Maximum Runtime	**895.198 ms**	1565.560 ms	单案例运行时间最大值
独立检查 Area/Delay	**25/25 一致**	—	不依赖综合器内部成本累计
独立逻辑验证	**13 穷举 + 12 抽样**	—	大规模抽样不能当成穷举证明

4.2 全部 25 个案例

表中 `Slack = Target − Delay`。全部 Slack 非负，`status=PASS` 且 `target_met=yes`。

Case	n	m	Area	Delay	Target	Slack	Runtime (ms)	Target Met
001	15	14	119,318	71	77	+6	181.369	Yes
002	14	14	18,918	60	61	+1	99.945	Yes
003	14	14	27,445	54	64	+10	86.705	Yes
004	13	10	7,940	55	57	+2	16.363	Yes
005	8	5	1,241	30	36	+6	26.594	Yes
006	12	12	11,002	41	60	+19	18.664	Yes
007	20	15	139,536	65	89	+24	895.198	Yes
008	12	12	15,586	56	62	+6	24.652	Yes
009	12	6	23,648	41	59	+18	36.427	Yes
010	19	8	4,771	34	61	+27	197.275	Yes
011	10	5	484	35	40	+5	32.032	Yes
012	10	10	5,760	25	40	+15	13.931	Yes
013	14	14	12,106	55	70	+15	43.381	Yes
014	20	8	11,648	61	72	+11	419.171	Yes
015	16	16	36,345	62	81	+19	159.299	Yes
016	10	5	4,206	46	47	+1	8.826	Yes
017	13	12	455	22	25	+3	8.137	Yes
018	12	12	1,384	29	32	+3	10.269	Yes
019	18	8	9,349	51	73	+22	104.069	Yes
020	8	8	776	30	35	+5	35.332	Yes
021	15	14	28,134	60	72	+12	129.295	Yes
022	8	16	1,336	31	34	+3	31.156	Yes
023	8	4	236	21	24	+3	6.830	Yes
024	18	8	162,143	67	83	+16	890.148	Yes
025	15	15	146,476	53	76	+23	278.383	Yes

4.3 同一 WSL 环境下的 Round 1 → Round 2 变化

指标	Round 1	Round 2	变化
PASS	25/25	25/25	功能保持正确
Target Met	12/25	**25/25**	**+13 个案例**
Area 总和	1,103,455	**790,243**	**−313,212（−28.38%）**
Median Runtime	5.455 ms	36.427 ms	×6.68
Maximum Runtime	95.293 ms	895.198 ms	×9.39

逐案例结论： 21/25 个案例的 Area 减少；4 个案例 Area 增加（`004`, `012`, `013`, `022`）；在 Delay 上有 22/25 个案例更短，其余持平，没有 Delay 变差的案例。由于多候选搜索和 Implicant 枚举，本轮 Runtime 明显增加。

Case	Round 1 Area → Round 2 Area	Δ Area (Round2 − Round1)	Round 1 Delay → Round 2 Delay
001	135,885 → 119,318	-16,567	71 → 71
002	21,835 → 18,918	-2,917	66 → 60
003	28,206 → 27,445	-761	66 → 54
004	7,713 → 7,940	+227	57 → 55
005	1,594 → 1,241	-353	36 → 30
006	14,272 → 11,002	-3,270	56 → 41
007	231,712 → 139,536	-92,176	89 → 65
008	15,924 → 15,586	-338	56 → 56
009	33,212 → 23,648	-9,564	56 → 41
010	9,394 → 4,771	-4,623	75 → 34
011	620 → 484	-136	47 → 35
012	4,157 → 5,760	+1,603	46 → 25
013	11,826 → 12,106	+280	67 → 55
014	20,808 → 11,648	-9,160	81 → 61
015	38,556 → 36,345	-2,211	76 → 62
016	5,008 → 4,206	-802	46 → 46
017	573 → 455	-118	26 → 22
018	2,933 → 1,384	-1,549	41 → 29
019	23,692 → 9,349	-14,343	76 → 51
020	824 → 776	-48	36 → 30
021	69,666 → 28,134	-41,532	72 → 60
022	1,286 → 1,336	+50	37 → 31
023	262 → 236	-26	35 → 21
024	180,822 → 162,143	-18,679	84 → 67
025	242,675 → 146,476	-96,199	71 → 53

有代表性的观测：`case_010` Delay 由 75 降到 34，`case_023` 从 35 降到 21；原本延迟未达标的全部 13 个案例均转为达标。面积方面，`case_025` 减少 96,199，`case_007` 减少 92,176，而 `case_012` 的 Area 增加 1,603，说明 Timing/Area Trade-off 不可能靠一个单调改善的等价变换来概括。

归因界限： Round 2 最终版是多个优化策略的组合；从这张对照表不能把某一案例的收益全部归因于 Two-level Shannon、Greedy Cover 或 Influence Ordering。单方法的净收益应通过关闭该特性、使用同一机器和相同输入进行 Ablation 才能严格测量。

5. 验证覆盖、证据链与可信度边界

1. **官方 Evaluator：** 25 个案例的 `status=PASS`，并提供 Area、Delay、Runtime、Target Met 等字段；判断是否满足 Target 必须读取 `target_met`，不能仅看 PASS。
2. **独立 Netlist Checker：** 本地 ZIP 的 `independent_crosscheck.csv` 显示 25/25 个 Area/Delay 与官方逐行完全相等；`verification_summary.txt` 显示 **13 个案例穷举输入赋值，12 个案例固定种子抽样**，

全部无问题。

3. **源码与证据版本：**本地 `synth_source_snapshot.cpp` 的哈希匹配冻结源码；原始 ZIP 保留了编译日志、Evaluator 日志、输出 Netlist 与输入案例的哈希清单。
4. **强度界限：** 12 个较大案例的第二套独立模拟是抽样，不可写成独立全表穷举；但老师的官方 Evaluator 在 25 个案例上均判定为 PASS。
5. **还没有证明的事：**Hidden Cases 的性能；全球最优 Area/Delay；任何输入的严格 Runtime/Memory 保证；官方课程成绩预测；任何尚未发生的个人或小组贡献。

6. Round 2 的局限性及向 Round 3 的演进

- **局部决策与全图目标仍存在差距。** Round 2 的 De Morgan 和展开接受标准侧重局部 Arrival Time；即使某路径还有很大的 Timing Slack，也可能因为局部判据过于严格而错失面积恢复机会。
- **函数的极性共享不足。**原版本没有 Complemented-edge BDD，也没有 Affine/XOR 子函数的专门识别；这些是后续 Round 3 的优化方向，而不是已实现特性。
- **候选时间预算只在候选之间检查。**一个需要很长时间的单候选可以超过目标预算；本地全部公开案例 PASS 不能证明未知测试下严格可中断。
- **公共数据过拟合风险。**本轮只在已公开的 25 个官方案例和相应历史记录上报告这组对比，没有提供同机上对隐藏案例的测试结果。
- **结果指标不是课程评分。** Area 总和并不自动等于分数；终值选择策略也只是项目自定义的排序规则。

阶段性结论： Round 2 成功解决 Round 1 的全部已发布 Delay Target 违规（12/25 → 25/25），并把 Area 总和从 1,103,455 降到 790,243。代价是更高的 Runtime。下一轮研究应把重点从“找到短路径表示”推进到“利用全图 Required Time / Slack，允许非关键路径变慢以恢复 Area”，这构成 Round 3 的主要研究动机。

7. 复现命令、版本边界与证据索引

```
BASE="/mnt/e/canfiles/Common courses study/大学通识/26-27 Fall/UFUG 2106 Discrete Math/assignment/
cd "$BASE/UFUG2106_Round2_Delivery"
bash run_round_local.sh /tmp/ufug2106_evaluator "$BASE/data/stage1"
```

证据 ZIP：`20261111_104026_Round2_local_results.zip`，包含 `official.csv`、`reference.csv`、`per_case_comparison.csv`、`independent_crosscheck.csv`、`verification_summary.txt`、`synth_source_snapshot.cpp`、`environment.txt`、`case_sha256.txt`、`build.log`、`evaluator.log` 及 25 个生成 Netlist。算法说明另存于 `UFUG2106_Round2_Delivery/docs/ALGORITHM_AND_MATH.md`。

版本边界：更早的 `intermediate_portfolio_21target.cpp` 取得过 21/25 Target Met、Area 904,009，属于 **Round 2 内部迭代**，不能拿它冒充这次本地复现的最终归档 `src/synth.cpp`。Round 3 的 Complemented Edges、Affine、Required-time Recovery 也不应追溯写到 Round 2。

参与声明：本次数据说明同一用户环境完成了本地复现，不据此虚构团队成员个人对算法实现或代码开发的贡献；总 Report 应如实写明 AI 辅助、实际参与及仍待完成的工作。